

“While” loops in Python

March 9, 2026

Administrative Notes

Reminder - quiz 2 is Wednesday, in class

Classwork 4 is due tonight, before midnight.

The schedule says I'm going to issue Classwork 5 today. That's going to be delayed until after Spring Break

You're working on Project 2; that's due on March 25 - the Wednesday after Spring Break

Loops

Loops are used to implement “iterative” control flow

- Where you do a set of things multiple times
 - Either a set number of times
 - Or until something is True or False

So far, we have covered one of the three types of loops* commonly used in Python:

```
for i in range(start, stop, step):
```

That loop is fine if you know how many times you want to execute the loop

- That is, you know the values of “start” and “stop”

If you don’t know that, you have to “break” out of the “for” loop early, which is not the best programming practice

***We’re not going to cover the third type of loop until after Spring Break**

Breaking out of a loop in the middle

(Caution: this is generally considered bad programming practice. Most of the time you can write a loop so that it doesn't break out in the middle. But sometimes it's necessary.)

Suppose that you don't know how while loops work, and you want to use a "for i" loop. It will run until something is True

- Like, say, Project 2. You give the user 6 guesses. If the user guesses the right number on the second guess, you don't want to make the user guess four more times before ending the program
 - That's just rude

So you use the "break" statement

The “break” statement in Python

an example of the break statement with "for i" loops in Python

```
from random import randint

number = randint(1,100)
print(number) #inserted to debug the code; remove when done!!
for i in range(0,6,1):
    guess = int(input("Please guess a number between 1 and 100"))
    if guess < number:
        print("Your guess was too low; guess again with a higher
number")
    elif guess > number:
        print ("Your guess was too high; guess again with a lower
number")
    else:
        print("Congratulations; you guessed correctly after ," i, "
guesses")
        break
if i >= 6:
    print("We're sorry you didn't win; please try again")
```

The “break” statement causes Python to stop executing the loop - to “break out of it” and immediately move to the first line of code after the loop.

This works, but there’s a better way to do this!!

The “better way” to do this? The “while loop”

- General syntax:

- `while (boolean-expression):`
 - `# code indented underneath the while statement`

- Semantics:

- At the beginning of the loop, evaluate the boolean-expression
 - If it's True, execute the code in the body of the loop
 - If it's False, skip the body of the loop and move on to the next line of code AFTER the loop - the next line not indented under “while”

This is the most general type of loop in Python

Why is this the “most general type of loop?”

- It can solve all problems that can be solved with looping
 - And do so elegantly, without resorting to a hack like the break statement
- You're not limited to executing a loop once for each item in a list; or a set number of times
- You may not know how many times your loop will be executed
 - It runs until the user types “quit”
 - It runs until there is no more data left in a file
 - It runs until something else happens
 - Your loop may run zero times
 - This is quite common for loops that check user input

Loops that run zero times

Suppose you ask the user to enter a positive integer

- You check to make sure that the user entered a positive integer
- While the user enters things other than a positive integer, you throw an error message and ask the user to try again
- If the user enters a positive integer the first time, your loop will never execute
 - This is desired behavior

Sentinel loops

A “sentinel” is a value that you are looking for, and take some action upon seeing

- E.g., write a loop that will run until the user enters some value, and then stops
- Here, “Q” is the “sentinel” value
-

Sentinels when reading from a file

If you're reading data from a file, the end of the file is marked with an empty line -
“”

- Since all other lines contain at least a newline character - “\n” - this is a unique way to tell the programmer that the file has no more data
- So you can use a loop with `readline()` and read one line at a time. Stop when you read an empty line

Infinite loops

An “infinite loop” is a loop that will literally run forever, unless you take some action outside of the program to stop it.

There are very rare instances when this is what you want - but normally, this is considered a “very bad thing”

It is very, very difficult to write an infinite loop using “for”

- You have to want to do it

But it is easy to write an infinite loop using “while”

- You can forget to increment a variable you’re using in the boolean expression
- You can design your boolean expression to be “not quite correct”
- Etc.

Examples of infinite loops

When your program has an infinite loop

How to tell that your program has an infinite loop

- It keeps printing output long after it should have stopped
 - if there's a `print()` statement in the loop
- It looks like nothing is happening for a long time
 - If there's no `print()` statement

What to do about it

- Hold down the Control key and hit c (Ctrl-c)
 - Either uppercase or lowercase C should work; lowercase is easier
- If you're on a Mac and that doesn't work, try Command-c
- If that doesn't work, you'll have to force IDLE to quit in the Operating System
 - Task Manager in Windows; Force Quit with Mac

Warning: with great power comes great responsibility

While loops are the most general type of loop in Python - they can solve any looping problem

BUT they are also the easiest loop to get wrong. And cause problems with your code that can be hard to debug

Why? Because it's easy to get into an infinite loop

- Variables in the boolean-expression are not automatically updated in the loop; you have to update them yourself
 - With both types of “for” loops, indices are updated automatically
- If you don't write the boolean-expression correctly, it may never be False

To summarize: why are while loops the most general type?

True statement:

- Any loop in Python can be written as a “while” loop
- Not every loop in Python can be written as a “for i” or “for each” loop
 - Loops where the number of times the loop gets executed depends on some unknown external influence